

Tag 8

Entscheidungen treffen & Funktionen

Python-Grundlagen-Schulung

Was du heute lernst

1. Wann benutze ich **if** und wann **while**?
2. Was ist eine **Funktion** – und warum brauche ich sie?
3. Was ist der Unterschied zwischen **print** und **return**?
4. Viel Übung: bekannte Aufgaben neu als Funktion schreiben

Teil 1

if oder while?

Erinnerst du dich an if?

```
if belegter_speicher > 90:  
    print("Achtung, Speicher fast voll!")
```

`if` heißt: "**Wenn etwas zutrifft, dann tu das.**"

Der Computer schaut sich die Bedingung **ein einziges Mal** an.

Erinnerst du dich an while?

```
while belegter_speicher > 90:  
    print("Räume auf...")
```

`while` heißt: "**Solange etwas zutrifft, wiederhole das.**"

Der Computer schaut sich die Bedingung **immer wieder** an – so lange, bis sie nicht mehr stimmt.

Der Unterschied als Bild

 if	 while
Einmal hinschauen	Immer wieder hinschauen
Reagieren	Warten und immer wieder prüfen
Weiterfahren	Bis sich etwas ändert

Bild 1: die Ampel (if)

Du kommst zur Ampel



Du schaust **einmal** hin



Grün? → Du fährst weiter

Rot? → Du hältst an



Fertig. Du schaust nicht nochmal hin.

Bild 2: die Bahnschranke (while)

Die Schranke ist unten



Du fragst: "Ist sie noch zu?" → Ja



Du wartest kurz



Du fragst nochmal: "Ist sie noch zu?" → Ja



... das geht so weiter, bis die Antwort "**Nein**" ist

Die einfache Frage, die du dir stellst

Passiert das nur EINMAL?

→ benutze `if`

Muss ich das WIEDERHOLEN, bis sich etwas ändert?

→ benutze `while`

Beispiel 1: Server-Speicher (einmalige Warnung)

Ich will **einmal** Bescheid geben, wenn der Speicher zu voll ist.

→ Das passiert nur einmal. **if** ist richtig.

```
belegter_speicher = 95  
  
if belegter_speicher > 90:  
    print("WARNUNG: Speicher fast voll!")
```

Beispiel 2: Server-Speicher (aufräumen)

Ich will **immer wieder** aufräumen, bis genug Platz frei ist.

→ Das wiederholt sich. **while** ist richtig.

```
belegter_speicher = 95

while belegter_speicher > 90:
    print(f"Speicher bei {belegter_speicher}%. Räume auf...")
    belegter_speicher = belegter_speicher - 3

print("Fertig aufgeräumt!")
```

Was passiert hier Schritt für Schritt?

1. belegter_speicher ist 95 → größer als 90 → Schleife startet
2. Ausgabe: "Speicher bei 95%..." → jetzt: $95 - 3 = 92$
3. 92 ist immer noch größer als 90 → Schleife läuft nochmal
4. Ausgabe: "Speicher bei 92%..." → jetzt: $92 - 3 = 89$
5. 89 ist **nicht mehr** größer als 90 → Schleife stoppt
6. Ausgabe: "Fertig aufgeräumt!"

Kurz zusammengefasst

if = einmal prüfen

while = wieder und wieder prüfen, bis Schluss ist

Jetzt übst du das an 4 kleinen Aufgaben im Colab-Notebook. 

Teil 2

Was ist eine Funktion?

Das Problem ohne Funktionen

Stell dir vor, du willst dreimal jemanden begrüßen:

```
print("Hallo Alex, willkommen!")  
print("Hallo Sabine, willkommen!")  
print("Hallo Max, willkommen!")
```

Du schreibst **fast denselben Satz** immer wieder. 😞

Was, wenn du den Satz später ändern willst? Dann musst du ihn an **jeder** Stelle ändern!

Die Lösung: eine Funktion

Stell dir eine Funktion wie eine **Maschine** vor:



Du wirfst etwas rein (z. B. einen Namen)



Die Maschine macht etwas damit



Es kommt etwas raus (oder wird angezeigt)

Du baust die Maschine nur einmal.

Danach benutzt du sie so oft du willst.

Eine Funktion bauen

```
def begruesse_user(username):  
    print(f"Hallo {username}, willkommen!")
```

- **def** → "ich baue jetzt eine Funktion" (def = definieren)
- **begruesse_user** → der Name, den ich mir ausdenke
- **(username)** → das, was ich reinwerfen kann
- **Einrückung** → alles, was zur Funktion gehört

⚠ Wichtig: Nur weil ich das schreibe, passiert **noch gar nichts!**

Die Funktion benutzen

```
def begruesse_user(username):  
    print(f"Hallo {username}, willkommen!")  
  
begruesse_user("Alex")  
begruesse_user("Sabine")  
begruesse_user("Max")
```

Erst wenn ich `begruesse_user("Alex")` schreibe,
wird die Maschine wirklich benutzt – mit "Alex" als Zutat.

Bild: Die Funktion als Maschine

"Alex" →  begrüesse_user  → "Hallo Alex, willkommen!"

"Sabine" →  begrüesse_user  → "Hallo Sabine, willkommen!"

Gleiche Maschine – unterschiedliche Zutat – unterschiedliches Ergebnis!

Warum ist das gut?

- ✓ Ich schreibe den Text nur **einmal**
- ✓ Will ich den Text ändern, ändere ich ihn nur an **einer** Stelle
- ✓ Mein Code wird **kürzer und übersichtlicher**

Das nennt man das **DRY-Prinzip**:

Don't Repeat Yourself – "Wiederhole dich nicht"



Kurze Pause

Teil 3

print vs. return

Die wichtigste Frage heute

Will ich das Ergebnis nur **anschauen**?

→ `print`

Will ich mit dem Ergebnis später **weiterarbeiten**?

→ `return`

print: nur anzeigen

```
def addiere_print(a, b):  
    print(a + b)  
  
addiere_print(5, 5)
```

Ausgabe auf dem Bildschirm: **10**

Aber: Diese 10 ist **weg**. Ich kann nicht mehr damit rechnen.

return: Ergebnis zurückgeben

```
def addiere_return(a, b):  
    return a + b  
  
ergebnis = addiere_return(5, 5)  
print(ergebnis)
```

`ergebnis` **speichert** jetzt die Zahl **10**.

Ich kann damit weiterarbeiten!

Weiterarbeiten – ein Beispiel

```
def addiere_return(a, b):  
    return a + b  
  
ergebnis = addiere_return(5, 5)  
print(ergebnis * 2)      # 20
```

Weil `ergebnis` die Zahl 10 gespeichert hat,
kann ich sie ganz normal weiterverwenden – z. B. mal 2 rechnen.

Mit `print` allein wäre das **nicht möglich** gewesen!

Bild: print vs. return

print = Ich rufe dir das Ergebnis laut zu.

Danach ist es weg. 

return = Ich gebe dir das Ergebnis auf einem Zettel.

Du kannst ihn behalten und weiterverwenden.  

Merksatz zum Einprägen

print zeigt.
return liefert.

💡 Falls deine Variable nach einem Funktionsaufruf leer bleibt (`None`),
hast du wahrscheinlich `return` vergessen!

Teil 4

Jetzt du: viel Übung!

Was du jetzt machst

Du schreibst Aufgaben, die du schon kennst, **neu als Funktion**.

Zum Beispiel:

- E-Mail-Adresse zusammenbauen
- Preis mit Mehrwertsteuer berechnen
- Temperatur bewerten (zu heiß / zu kalt / okay)
- Passwort vergleichen

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
def berechne_brutto(netto, mwst=19):  
    faktor = 1 + (mwst / 100)  
    return netto * faktor  
  
print(berechne_brutto(100))      # 119.0  
print(berechne_brutto(100, 7))  # 107.0
```

`mwst=19` ist ein **Standardwert**.

Gebe ich keinen zweiten Wert an, wird automatisch 19 genutzt.

Gebe ich einen an (z. B. 7), wird der genutzt.

Zum Schluss

Typische Anfängerfehler

Fehler 1: Einrückung vergessen

```
def sag_hallo():  
print("Hallo!")      # ✗ Fehler!
```

Der Code **innerhalb** der Funktion muss **engerückt** sein.
Sonst weiß Python nicht, dass die Zeile zur Funktion gehört.

Fehler 2: Variable "verschwindet"

```
def berechne_quadrat(zahl):  
    ergebnis = zahl * zahl  
    return ergebnis  
  
x = berechne_quadrat(4)  
print(x)          #  funktioniert, zeigt 16  
  
print(ergebnis)   #  Fehler! ergebnis kennt Python hier nicht
```

Eine Variable, die **in** einer Funktion entsteht,
existiert **nur dort** – nicht außerhalb.

Was du heute gelernt hast

- ✓ **if** = einmal prüfen, **while** = wiederholt prüfen
- ✓ Eine **Funktion** ist eine wiederverwendbare Code-Maschine
- ✓ **print** zeigt an, **return** liefert ein Ergebnis zum Weiterarbeiten

Morgen geht's weiter mit: Dictionaries 🎉